



SOFTWARE TESTING – LAB 1

Testing Fundamentals through Effective Prompts

Họ và tên: Võ Văn Thiên Bảo

MSSV: DE190619

Lớp: SE20A09

Ngày nộp: 19/05/2026

Công cụ AI dùng: ChatGPT

SOFTWARE TESTING – LAB 1

Testing Fundamentals through Effective Prompts

Contents

1. Nhiệm vụ 1: Why is testing necessary?	4
1.1. Prompt	4
1.2. Câu trả lời AI	4
1.2.1. Why is testing necessary?	4
1.2.2. Phân biệt Error, Defect, Failure và Root Cause	5
1.2.3 Quality Assurance (QA) vs Testing	6
1.2.4. Rủi ro và Chi phí khi không kiểm thử đủ	6
1.3. Nhận xét kết quả trả lời của AI	7
2. Nhiệm vụ 2: What is testing?	8
2.1. Prompt:	8
2.1.1. Zero-shot	8
2.1.2. Few-shot	8
2.2. Câu trả lời của AI:	8
2.2.1. Zero-shot	8
2.2.2. Few-shot	9
2.3. Nhận xét kết quả trả lời của AI	11
3. Nhiệm vụ 3: 7 principles of software testing	12
3.1. Prompt:	12
3.1.1. Prompt 1:	12
3.1.2. Prompt 2:	12
3.1.3. Prompt 3:	12
3.2. Câu trả lời của AI	12
3.2.1. Testing shows the presence of defects, not their absence (Kiểm thử chứng minh phần mềm có lỗi, chứ không thể chứng minh phần mềm hết lỗi)	12
3.2.2. Exhaustive testing is impossible (Không thể kiểm thử cạn kiệt mọi trường hợp)	13

3.2.3. Early testing saves time and money (Kiểm thử càng sớm, càng tiết kiệm chi phí)	13
3.2.4. Defects cluster together (Lỗi thường có xu hướng tụ tập lại một chỗ)	14
3.2.5. Beware of the pesticide paradox (Cẩn thận với nghịch lý thuốc trừ sâu)	14
3.2.6. Testing is context dependent (Kiểm thử phụ thuộc vào ngữ cảnh)	15
3.2.7. Absence-of-defects fallacy (Sự nguy hiểm về việc vắng mặt lỗi)	15
3.3. Nhận xét kết quả câu trả lời AI	16
4. Nhiệm vụ 4: Test activities & roles	17
4.1. Prompt:	17
4.2. Câu trả lời AI	17
4.2.1. Test planning (Lập kế hoạch)	17
4.2.2. Test monitoring and control (Giám sát và kiểm soát)	17
4.2.3. Test analysis (Phân tích)	18
4.2.4. Test design (Thiết kế)	18
4.2.5. Test implementation (Triển khai)	18
4.2.6. Test execution (Thực thi)	18
4.2.7. Test completion (Hoàn tất)	19
4.3. Nhận xét kết quả trả về của AI	19
5. Nhiệm vụ 5: Essential skills của tester	20
5.1. Prompt:	20
5.1.1. Prompt lần 1:	20
5.1.2. Prompt lần 2:	20
5.3. Câu trả lời AI	20
5.4. Nhận xét kết quả trả về của AI	21
5.5. Tổng kết kỹ năng quan trọng em rút ra được	21
6. Phần kết luận	22
7. Tài liệu tham khảo	22

Thông tin tác giả:

- Họ và tên: Võ Văn Thiên Bảo
- MSSV: DE190619
- Lớp: SE20A09
- Ngày nộp: 19/05/2026
- Công cụ AI dùng: ChatGPT
- Link Github: <https://github.com/T-618/swt301-de190619-vovanthienbao.git>

1. Nhiệm vụ 1: Why is testing necessary?

1.1. Prompt

Bạn là một test manager có chứng chỉ ISTQB với hơn 10 năm kinh nghiệm trong việc kiểm thử phần mềm. Tôi là sinh viên đang học môn software testing. Tôi cần hiểu rõ bản chất cốt lõi của phần mềm dựa trên câu hỏi: Why is testing necessary? Hãy phân biệt 4 khái niệm: error, defect, failure, root cause. Sau đó, giải thích ngắn gọn về quality assurance vs testing; rủi ro & chi phí khi không kiểm thử đủ. Hãy lấy ví dụ thực tế áp dụng cho hệ thống Booking Hotel Online. Đối với phần phân biệt 4 khái niệm, hãy trình bày theo format bảng 3 cột (Khái niệm | Định nghĩa ngắn gọn dưới 30 từ | Ví dụ thực tế ở Booking Hotel Online).

1.2. Câu trả lời AI

1.2.1. Why is testing necessary?

Phần mềm được tạo ra bởi con người, và con người thì luôn có nguy cơ mắc sai lầm. Những sai lầm này tạo ra khiếm khuyết trong phần mềm. Nếu không được kiểm thử để phát hiện kịp thời, các khiếm khuyết này sẽ gây ra sự cố khi phần mềm đến tay người dùng, dẫn đến:

- Tổn thất về tài chính (đền bù, sửa chữa).
- Lãng phí thời gian.
- Tổn hại nghiêm trọng đến uy tín của doanh nghiệp.
- Thậm chí gây nguy hiểm đến tính mạng (trong các hệ thống y tế, hàng không).

Do đó, kiểm thử là công cụ quản lý rủi ro thiết yếu, giúp đo lường và cải thiện chất lượng phần mềm trước khi phát hành.

1.2.2. Phân biệt Error, Defect, Failure và Root Cause

Dưới đây là sự phân biệt 4 khái niệm cốt lõi của ISTQB, áp dụng cho hệ thống Web/App đặt phòng khách sạn online:

Khái niệm	Định nghĩa	Ví dụ
Error (Lỗi/Nhầm lẫn)	Hành động sai lầm của con người sinh ra kết quả sai. Thường do một mới, thiếu kinh nghiệm.	Lập trình viên nhầm lẫn logic tính toán khi code tính năng giảm giá 20% cho khách hàng VIP.
Defect (Khiếm khuyết/Bug)	Sự không hoàn hảo, sai sót trong mã nguồn hoặc tài liệu, được sinh ra từ Error.	Trong source code, biến <code>thanh_tien</code> thay vì nhân 0.8 lại bị trừ đi đúng 20 VNĐ.
Failure (Sự cố)	Sự sai lệch của hệ thống so với mong đợi khi một Defect bị kích hoạt lúc thực thi.	Khách VIP đặt phòng giá 1.000.000 VNĐ, hệ thống lại yêu cầu thanh toán 999.980 VNĐ thay vì 800.000 VNĐ.

Root cause (Nguyên nhân gốc rễ)	Nguyên nhân sâu xa, cơ bản nhất dẫn đến Error. Triệt tiêu nó giúp ngăn ngừa lỗi tương tự.	Tài liệu yêu cầu (Requirement) ghi không rõ ràng là "giảm 20%" hay "giảm 20 VNĐ", làm dev hiểu sai.
---	---	---

1.2.3 Quality Assurance (QA) vs Testing

Dù thường bị đánh đồng, trong chuẩn ISTQB đây là hai hoạt động khác biệt và bổ trợ cho nhau:

- **Quality Assurance (Đảm bảo chất lượng):** Hướng đến **Quy trình (Process)**. QA tập trung vào việc phòng ngừa lỗi (Defect Prevention) bằng cách thiết lập các tiêu chuẩn, quy trình làm việc (VD: quy trình review code, chuẩn hóa tài liệu). QA đảm bảo chúng ta đang *"xây dựng phần mềm đúng cách"*.
- **Testing (Kiểm thử):** Hướng đến **Sản phẩm (Product)**. Testing là các hoạt động thực thi hệ thống hoặc kiểm tra tài liệu để tìm ra lỗi (Defect Identification) đã tồn tại. Testing đảm bảo chúng ta *"đã xây dựng đúng phần mềm theo yêu cầu"*.

1.2.4. Rủi ro và Chi phí khi không kiểm thử đủ

Việc cắt giảm thời gian kiểm thử sẽ dẫn đến những hậu quả dây chuyền:

- **Chi phí sửa lỗi tăng theo cấp số nhân (Quy tắc số 10):** Một lỗi được phát hiện ở khâu lấy yêu cầu tốn rất ít chi phí để sửa. Nhưng nếu lỗi tính sai tiền phòng lọt ra môi trường thật (Production), công ty không chỉ tốn nguồn lực

hotfix, dừng server, mà còn phải xuất tiền túi đền bù khoản chênh lệch cho hàng ngàn khách hàng đã đặt phòng.

- **Rủi ro kinh doanh lớn:** Ví dụ, nếu kiểm thử không bao phủ được kịch bản "2 người book cùng 1 phòng cùng lúc" (Race condition), lỗi Overbooking sẽ xảy ra. Khách hàng tới khách sạn trong đêm nhưng không có phòng sẽ cực kỳ phẫn nộ, họ sẽ đánh giá 1 sao trên các nền tảng OTA (Agoda, Booking...), tạo khủng hoảng truyền thông và tàn phá uy tín thương hiệu khách sạn.

1.3. Nhận xét kết quả trả lời của AI

Câu trả lời của AI khá dễ hiểu và bám khá sát lý thuyết theo chuẩn ISTQB. AI tuân thủ đúng format, áp dụng ví dụ dễ hiểu (ở đây là hệ thống Hotel Booking Online) một cách xuyên suốt và logic (từ error của dev dẫn đến defect trong code và failure khi khách thanh toán). Điều này giúp các khái niệm giúp cho người mới dễ tiếp cận.

2. Nhiệm vụ 2: What is testing?

2.1. Prompt:

2.1.1. Zero-shot

Bạn là giảng viên bộ môn Software Testing. Hãy định nghĩa "Testing là gì?". Tiếp theo, hãy so sánh điểm khác biệt giữa Testing với Debugging, và giữa Verification với Validation. Trình bày ngắn gọn, gạch đầu dòng rõ ý, dưới 250 từ.

2.1.2. Few-shot

Bạn là chuyên gia đào tạo ISTQB Foundation với hơn 10 năm kinh nghiệm trong việc giảng dạy. Tôi đang ôn tập các khái niệm nền tảng của kiểm thử phần mềm dựa trên câu hỏi: What is testing? Hãy định nghĩa Testing. Sau đó so sánh Testing vs Debugging, và Verification vs Validation. Trình bày dưới dạng 2 bảng so sánh riêng biệt. Trả lời súc tích. Ví dụ cấu trúc bảng Verification vs Validation: | Tiêu chí | Verification | Validation | | Mục tiêu | Đánh giá xem hệ thống có được xây dựng đúng cách không (Are we building the product right?) | Đánh giá xem hệ thống có đáp ứng đúng nhu cầu người dùng không (Are we building the right product?) |

2.2. Câu trả lời của AI:

2.2.1. Zero-shot

a. Testing là gì? Là khâu kiểm tra và đánh giá một hệ thống để xác minh xem nó có thỏa mãn các yêu cầu đã đặt ra ban đầu hay không. Mục đích cốt lõi là chỉ ra khoảng cách giữa kết quả thực tế (Actual result) và kết quả kỳ vọng (Expected result).

b. Phân biệt Testing và Debugging:

- **Testing:** Nhằm mục đích bộc lộ và chỉ điểm lỗi (Defect Identification). Thường do Tester phụ trách để tìm ra sự sai lệch so với yêu cầu.
- **Debugging:** Nhằm mục đích chẩn đoán nguyên nhân và sửa chữa lỗi (Defect Resolution). Do Developer phụ trách để khoanh vùng dòng code hỏng và khắc phục.

c. Phân biệt Verification và Validation:

- **Verification (Xác minh):** Kiểm tra xem việc xây dựng phần mềm có bám sát thiết kế và quy trình không ("Làm sản phẩm đúng cách chưa?"). Áp dụng nhiều qua việc review tài liệu.
- **Validation (Xác nhận):** Đánh giá xem phần mềm có giải quyết đúng bài toán của người dùng cuối không ("Làm đúng sản phẩm chưa?"). Áp dụng qua việc chạy thử nghiệm thực tế.

2.2.2. Few-shot

a. Định nghĩa Testing Theo chuẩn ISTQB, Kiểm thử (Testing) là một quy trình bao trùm toàn bộ vòng đời phát triển, chứa cả các hoạt động tĩnh và động, nhằm:

- Đánh giá sản phẩm phần mềm (tài liệu, mã nguồn).
- Tìm kiếm các khiếm khuyết (Defects).
- Đảm bảo sản phẩm đáp ứng các yêu cầu chỉ định và phù hợp với mục đích sử dụng thực tiễn của người dùng.

b. So sánh Testing và Debugging

Tiêu chí	Testing (Kiểm thử)	Debugging (Gỡ lỗi)
Mục đích	Tìm kiếm và kích hoạt các khiếm khuyết (Bug) để chứng minh phần mềm có lỗi.	Truy vết nguyên nhân gốc (Root cause) và tiến hành sửa đổi mã nguồn để fix lỗi.
Người đảm nhiệm	Đa số do Tester / QA thực hiện.	Do Developer (Lập trình viên) đảm nhiệm.
Cách thức	Thực thi phần mềm hoặc đọc kiểm tra tài liệu để tìm dấu hiệu bất thường.	Đọc luồng logic của code, khoanh vùng lỗi và thay đổi cấu trúc hệ thống.

c. So sánh Verification và Validation

Tiêu chí	Verification (Xác minh)	Validation (Xác nhận)
Câu hỏi cốt lõi	Are we building the product right? (Chúng ta có đang làm đúng theo quy trình/thiết kế?)	Are we building the right product? (Chúng ta có tạo ra đúng thứ người dùng cần?)
Đối tượng	Các tài liệu trung gian (Requirement, Design, Architecture).	Sản phẩm phần mềm cuối cùng đã chạy được.
Phương pháp	Kiểm thử tĩnh (Static Testing): Review, Inspection...	Kiểm thử động (Dynamic Testing): Chạy kịch bản Black-box, White-box...

2.3. Nhận xét kết quả trả lời của AI

Em thấy prompt few-shot hiệu quả hơn vì AI trả lời đúng format, có bám sát lí thuyết theo chuẩn ISTQB và có ví dụ cụ thể liên quan đến hệ thống Booking Hotel Online hơn so với prompt zero-shot.

3. Nhiệm vụ 3: 7 principles of software testing

3.1. Prompt:

3.1.1. Prompt 1:

Bạn là Senior QA Engineer. Hãy giải thích 2 nguyên tắc kiểm thử: (1) Testing shows the presence of defects, not their absence và (2) Exhaustive testing is impossible. Liệt kê đánh số, mỗi nguyên tắc kèm theo một ví dụ chung chung trong ngành IT. Giải thích dễ hiểu cho người mới bắt đầu học IT.

3.1.2. Prompt 2:

Bạn là Lead Tester. Tôi đang tham gia dự án phần mềm "CinePlex" - hệ thống web/app đặt vé rạp chiếu phim. Hãy giải thích 3 nguyên tắc: (3) Early testing saves time and money, (4) Defects cluster together, (5) Pesticide paradox. Liệt kê đánh số. Với mỗi nguyên tắc, hãy áp dụng trực tiếp 1 ví dụ thực tế có thể xảy ra trong dự án CinePlex (ví dụ: tính năng chọn ghế, cổng thanh toán).

3.1.3. Prompt 3:

Cùng bối cảnh dự án "CinePlex" ở trên. Hãy giải thích 2 nguyên tắc cuối cùng: (6) Testing is context dependent và (7) Absence-of-defects fallacy. Vẫn giữ định dạng liệt kê đánh số và cung cấp ví dụ áp dụng trực tiếp cho app rạp phim này.

3.2. Câu trả lời của AI

(Kết quả từ Prompt 1)

3.2.1. Testing shows the presence of defects, not their absence (Kiểm thử chứng minh phần mềm có lỗi, chứ không thể chứng minh phần mềm hết lỗi)

- **Giải thích:** Việc test giúp chúng ta khai ra các lỗi đang tồn tại. Tuy nhiên, dù bạn có chạy hàng vạn test case thành công thì cũng không thể kết luận phần mềm đã "sạch bóng" 100% bug. Kiểm thử chỉ giúp hạ thấp rủi ro lỗi xảy ra trên môi trường thật, chứ không phải là lời cam kết hoàn hảo.
- **Ví dụ chung:** Khi test tính năng Đăng nhập, bạn nhập đủ kiểu: sai mật khẩu, bỏ trống, chèn mã độc... và mọi thứ đều chạy đúng. Nhưng bạn không thể lường trước trường hợp user đăng nhập bằng mạng 3G yếu rồi liên tục ấn nút F5. Lỗi vẫn luôn có thể ẩn nấp ở một góc nào đó.

3.2.2. Exhaustive testing is impossible (Không thể kiểm thử cạn kiệt mọi trường hợp)

- **Giải thích:** Việc test toàn bộ mọi trường hợp đầu vào, mọi luồng xử lý và điều kiện môi trường là chuyện bất khả thi và lãng phí thời gian. Thay vì cố gắng test mọi thứ, Tester phải dùng kỹ năng phân tích rủi ro để chọn ra những kịch bản quan trọng nhất cần ưu tiên.
- **Ví dụ chung:** Một app Máy tính (Calculator) đơn giản có phép cộng 2 số. Tổ hợp các con số từ hàng âm đến hàng tỷ là vô hạn. Không ai ngồi gõ tay tất cả các số đó để test. Người ta chỉ chọn các giá trị đại diện (số 0, số âm, số dương siêu lớn) để kiểm tra.

(Kết quả từ Prompt 2)

3.2.3. Early testing saves time and money (Kiểm thử càng sớm, càng tiết kiệm chi phí)

- **Giải thích:** Các hoạt động QA (đọc yêu cầu, review thiết kế) nên bắt đầu ngay từ những ngày đầu của dự án. Sửa một lỗi logic trên tài liệu giấy tốn ít công

sức hơn rất nhiều so với việc để lỗi đó lọt vào giai đoạn code, hoặc tệ hơn là khi ứng dụng đã tới tay người dùng.

- **Ví dụ tại CinePlex:** Tài liệu BA viết "Người dùng có 5 phút để giữ ghế". Nếu QA đọc sớm, sẽ thắc mắc ngay: "Nếu khách tắt ngang app thì ghế đó bị giam luôn 5 phút à?". Nhờ phát hiện sớm, luồng logic được sửa lại ngay lập tức. Nếu để code xong mới phát hiện, rạp phim sẽ mất doanh thu vì "ghế ảo" và Dev phải đập đi xây lại luồng xử lý.

3.2.4. Defects cluster together (Lỗi thường có xu hướng tụ tập lại một chỗ)

- **Giải thích:** Theo nguyên lý 80/20, đa số các bug nghiêm trọng thường nằm tập trung ở một vài tính năng cốt lõi hoặc phức tạp nhất của phần mềm. Nhận thức được điều này giúp QA dồn lực vào đúng chỗ thay vì test dàn trải.
- **Ví dụ tại CinePlex:** Trong app rạp phim, chắc chắn module "Thanh toán & Áp mã khuyến mãi" là nơi sinh ra nhiều bug nhất. Vì ở đây xử lý logic cực khó: trừ tiền ví điện tử, tính điểm thành viên, áp dụng combo bắp nước... Do đó, team QA sẽ phải dành 80% công lực để test đi test lại khu vực nhạy cảm này.

3.2.5. Beware of the pesticide paradox (Cẩn thận với nghịch lý thuốc trừ sâu)

- **Giải thích:** Nếu chỉ dùng một loại thuốc trừ sâu, sâu sẽ nhờn thuốc. Tương tự, nếu tester chỉ chạy lặp đi lặp lại một kịch bản test cũ, hệ thống sẽ "miễn nhiễm" và không lòi ra bug mới. QA phải liên tục tư duy, đổi mới kịch bản và dữ liệu test.
- **Ví dụ tại CinePlex:** Kịch bản thanh toán MoMo lúc đầu chỉ test "Đủ tiền" và "Thiếu tiền", chạy mãi luôn PASS. Nhưng vào dịp Lễ, lượng truy cập tăng vọt, khách trừ tiền rồi nhưng app CinePlex lại quay mòng mòng không nhả vé

do mạng chập chờn. Team QA phải lập tức "đổi thuốc" bằng cách bổ sung ngay các kịch bản về Network Timeout vào bộ test định kỳ.

(Kết quả từ Prompt 3)

3.2.6. Testing is context dependent (Kiểm thử phụ thuộc vào ngữ cảnh)

- **Giải thích:** Không có một phương pháp hay chiến lược kiểm thử nào áp dụng đúng cho mọi dự án. Tùy thuộc vào loại phần mềm (ngân hàng, game, thương mại điện tử) mà mức độ ưu tiên, kỹ thuật test và mức độ rủi ro sẽ hoàn toàn khác nhau.
- **Ví dụ tại CinePlex:** Chiến lược test app rạp phim (CinePlex) sẽ tập trung cực mạnh vào Performance (hiệu năng tải trang) và Concurrency (xử lý đồng thời). Vì mỗi khi có phim bom tấn (như Avengers) mở bán, hàng ngàn người sẽ đổ xô vào giành chung một cái ghế đẹp. Trong khi đó, nếu test một website giới thiệu doanh nghiệp bình thường thì không cần tập trung quá nhiều vào yếu tố này.

3.2.7. Absence-of-defects fallacy (Sự nguy hiểm về việc vắng mặt lỗi)

- **Giải thích:** Một phần mềm không có bug (chạy đúng kỹ thuật 100%) chưa chắc đã là một phần mềm thành công nếu nó không đáp ứng đúng nhu cầu và mong đợi của người dùng cuối. Xây dựng sai sản phẩm thì dù không có bug cũng trở nên vô dụng.
- **Ví dụ tại CinePlex:** Team Dev và QA tự hào vì app CinePlex chạy mượt mà, không crash, thanh toán chính xác. Tuy nhiên, giao diện chọn ghế lại dùng màu sắc quá tối, nút bấm quá nhỏ khiến khách hàng bấm nhầm liên tục, hoặc bắt người dùng tạo tài khoản qua 10 bước phức tạp. Kết quả: Khách bỏ sang

app đối thủ. Phần mềm hoàn hảo về mặt code nhưng lại thất bại về mặt kinh doanh.

3.3. Nhận xét kết quả câu trả lời AI

Khi yêu cầu "ví dụ chung chung" như ở Prompt 1, AI đưa ra các ví dụ kinh điển (Calculator, Login) dễ hiểu cho người mới học. Nhưng khi đưa bối cảnh "Dự án CinePlex" ở Prompt 2 và 3, câu trả lời lại trở nên rõ ràng và sát với thực tế hơn. Các ví dụ như "giữ ghế 5 phút", "thanh toán MoMo bị timeout" hay "giành ghế xem phim bom tấn" giúp người mới có thể hình dung rõ ràng công việc thực tế của một Tester thay vì chỉ học lý thuyết.

4. Nhiệm vụ 4: Test activities & roles

4.1. Prompt:

Đóng vai một Test Manager nhiều năm kinh nghiệm. Bạn đang quản lý khâu kiểm thử cho dự án CinePlex (đặt vé xem phim). Hãy trình bày 7 hoạt động chính trong quy trình test (Test process theo chuẩn ISTQB): planning, monitoring & control, analysis, design, implementation, execution, completion. Viết theo phong cách kể chuyện ngôi thứ nhất (Tôi). Mỗi hoạt động viết tối đa 2 câu, đồng thời chỉ ra rõ đầu ra (artifact/testware) của hoạt động đó là gì. Trình bày dạng danh sách đánh số.

4.2. Câu trả lời AI

4.2.1. Test planning (Lập kế hoạch)

Tôi bắt đầu bằng việc xác định mục tiêu, phạm vi và rủi ro của hệ thống CinePlex, sau đó lên lịch trình và phân bổ nguồn lực QA cho các tính năng quan trọng như chọn ghế, thanh toán.

Đầu ra: Test plan.

4.2.2. Test monitoring and control (Giám sát và kiểm soát)

Hàng ngày, tôi theo dõi tiến độ chạy test và số lượng bug sinh ra ở cổng thanh toán để kịp thời điều chỉnh nhân lực nếu module này gặp rủi ro trễ deadline.

Đầu ra: Test progress report, defect status report.

4.2.3. Test analysis (Phân tích)

Dựa trên tài liệu đặc tả của BA, tôi và team phân tích các điều kiện kinh doanh (ví dụ: tuổi được xem phim 18+, quy định áp dụng voucher) để xác định xem cái gì cần phải test.

Đầu ra: Test conditions.

4.2.4. Test design (Thiết kế)

Từ các điều kiện đã phân tích, tôi hướng dẫn team phác thảo ra các kịch bản chi tiết, ví dụ như kịch bản khách hàng nhập mã giảm giá hết hạn hoặc thanh toán bị timeout.

Đầu ra: Test cases, test data.

4.2.5. Test implementation (Triển khai)

Chúng tôi tiến hành cài đặt môi trường test (server, database ảo), tạo sẵn các tài khoản user và chuẩn bị các mã voucher giả lập để chuẩn bị sẵn sàng cho việc chạy test.

Đầu ra: Test procedures, test scripts, test environment.

4.2.6. Test execution (Thực thi)

Team của tôi bắt đầu thao tác trực tiếp trên app CinePlex, chạy theo các case đã viết, so sánh kết quả thực tế với kỳ vọng và log bug lên hệ thống Jira cho đội Dev.

Đầu ra: Test execution log, defect reports.

4.2.7. Test completion (Hoàn tất)

Sau khi kiểm thử hoàn thành, tôi tổng hợp kết quả, đánh giá chất lượng hệ thống, bàn giao lại các bộ test tự động và tổ chức họp rút kinh nghiệm để làm tốt hơn ở dự án sau. Ngoài ra, các testware quan trọng được lưu lại để dùng cho lần kiểm thử sau.

Đầu ra: Test completion report, archived testware.

4.3. Nhận xét kết quả trả về của AI

Cách yêu cầu AI đóng vai ("Role-play") và xưng "Tôi" giúp các quy trình lý thuyết khô khan của ISTQB trở nên sinh động như một câu chuyện thực tế. Qua đó, mình phân biệt rõ được vai trò: Test Manager sẽ thiên về Planning, Monitoring và Completion, trong khi Tester sẽ trực tiếp làm nhiều ở khâu Analysis, Design và Execution.

5. Nhiệm vụ 5: Essential skills của tester

5.1. Prompt:

5.1.1. Prompt lần 1:

Bạn là IT Career Coach năm 2026. Tôi là sinh viên ngành kỹ thuật phần mềm theo định hướng Kỹ sư cầu nối (BrSE) thị trường Nhật Bản, đang học tiếng Nhật và môn Software Testing. Hãy phân tích 3 kỹ năng cốt lõi (Essential skills & good practices) quan trọng nhất mà một Tester/BrSE cần trang bị trong năm 2026 để làm việc hiệu quả với các dự án outsource. Hãy viết với format gạch đầu dòng rõ ràng cùng với đưa ra các ví dụ như: Kỹ năng giao tiếp, giải quyết sự cố bất đồng ngôn ngữ...

5.1.2. Prompt lần 2:

Câu trả lời trên vẫn còn hơi chung chung. Hãy viết lại, tập trung mạnh hơn vào: (1) Tư duy phản biện (Critical thinking) khi phân tích requirement của khách hàng, (2) Kỹ năng làm việc nhóm theo mô hình Agile (Whole-team approach). Yêu cầu trình bày lại dưới dạng bảng (Kỹ năng | Tầm quan trọng năm 2026 | Cách rèn luyện) và giới hạn dưới 200 từ.

5.3. Câu trả lời AI

Kỹ năng	Tầm quan trọng năm 2026 (Thị trường Nhật Bản)	Cách rèn luyện
1. Tư duy phản biện (Critical thinking)	Khách Nhật thường có Requirement khắt khe nhưng đôi khi truyền đạt ẩn ý. Tester/BrSE cần tư duy phản biện để khơi gợi đúng nhu cầu thực sự	Luôn đặt câu hỏi "Tại sao?", "Chuyện gì xảy ra nếu...?". Phân tích kỹ tài liệu đặc tả và thiết kế test

	(Honne), tránh làm sai lệch sản phẩm.	case bao phủ cả các Edge cases.
2. Làm việc nhóm Agile (Whole-team approach)	Dự án outsource đòi hỏi tốc độ cao. BrSE không chỉ dịch tài liệu mà phải cùng Dev và QA chịu trách nhiệm chất lượng ngay từ đầu (Shift-left testing).	Xóa bỏ tư duy "Dev làm, QA test". Chủ động tương tác đa chiều, thành thạo Jira/Confluence và tối ưu các buổi Daily Scrum.
3. Giao tiếp & Xử lý bất đồng ngôn ngữ	Cốt lõi của outsource là rào cản ngôn ngữ/văn hóa. BrSE là cầu nối giải quyết triệt để các hiểu lầm kỹ thuật giữa đội ngũ Việt Nam và khách hàng.	Luyện tiếng Nhật chuyên ngành (IT/Testing). Tuân thủ nghiêm ngặt văn hóa Ho-Ren-So (Báo cáo - Liên lạc - Bàn bạc).

5.4. Nhận xét kết quả trả về của AI

Ở lần đầu, AI trả lời còn khá chung chung. Nhờ bước tinh chỉnh ở Prompt 2 (ép AI xoáy sâu vào Tư duy phản biện, làm việc nhóm Agile, quy định định dạng bảng và giới hạn dưới 200 từ), kết quả trả về rõ ràng hơn và bám sát thực tế thị trường Nhật Bản. AI còn biết đưa ra các thuật ngữ chuyên sâu liên quan như văn hóa "Ho-Ren-So", tìm kiếm "Honne" (nhu cầu thực sự) hay "Shift-left testing".

5.5. Tổng kết kỹ năng quan trọng em rút ra được

Em nhận ra vai trò của một Tester/BrSE tại thị trường Nhật Bản năm 2026 không chỉ dừng lại ở việc tìm lỗi hay dịch thuật. Kỹ năng đắt giá nhất chính là Tư duy phản biện để thực sự thấu hiểu ý định của khách hàng (vượt qua văn hóa giao tiếp gián tiếp của người Nhật) và kỹ năng Whole-team approach để gắn kết team Dev Việt

Nam với đối tác. AI đã làm rất tốt việc đóng vai một IT Career Coach, giúp em định hình được lộ trình rèn luyện sát với thực tế ngành outsource thay vì chỉ học lý thuyết kiểm thử khô khan. Việc tinh chỉnh prompt lần 2 yêu cầu cụ thể về định dạng bảng và giới hạn từ ngữ đã giúp câu trả lời súc tích, đi thẳng vào trọng tâm hơn rất nhiều.

6. Phần kết luận

Qua bài Lab này, em đã nắm vững được những viên gạch nền tảng của ngành Software Testing. Em hiểu rằng kiểm thử không phải là công việc đơn thuần chạy chương trình rồi đợi lỗi xuất hiện, mà là một quy trình kỹ thuật nghiêm ngặt từ lập kế hoạch, phân tích rủi ro đến thực thi và báo cáo. Các khái niệm dễ nhầm lẫn như Error - Defect - Failure hay Testing - QA đều được làm sáng tỏ khi đặt vào bối cảnh dự án thực tế. Em cũng nhận thức sâu sắc về 7 nguyên tắc kiểm thử, đặc biệt là nguyên lý "Exhaustive testing is impossible" giúp em định hình tư duy test dựa trên rủi ro.

Bên cạnh đó, em học được nghệ thuật "Prompt Engineering" để biến AI thành một người thầy hỗ trợ học tập hiệu quả. Việc áp dụng công thức R-C-T-F-E (Role - Context - Task - Format - Example) giúp em điều khiển AI trả lời đúng trọng tâm, cấu trúc đẹp và gắn liền với thực tiễn thay vì trả về những đoạn văn lý thuyết sáo rỗng. Em nhận ra kỹ năng đặt câu hỏi cho AI trong tương lai cũng quan trọng không kém kỹ năng đọc hiểu code.

7. Tài liệu tham khảo

1. Foundations of Software Testing ISTQB Certification, 4th edition
<https://www.istqb.org/>
2. Prompt Engineering Guide
<https://www.promptingguide.ai/>